

Design and Analysis of Algorithms Programs for Turbo C

1. Binary Search

```
#include<stdio.h>

#include<conio.h>

void main(){

    int a[10], n, i, low, high, mid, key, found=0;

    clrscr();

    printf("Enter number of elements: ");

    scanf("%d",&n);

    printf("Enter sorted elements:\n");

    for(i=0;i<n;i++){

        scanf("%d",&a[i]);

    }

    printf("Enter element to search: ");

    scanf("%d",&key);

    low=0;

    high=n-1;

    while(low<=high){

        mid=(low+high)/2;

        if(a[mid]==key){

            found=1;

            break;

        }

        else if(a[mid]<key){

            low=mid+1;

        }

        else{

            high=mid-1;

        }

    }

}
```

```

if(found)

    printf("Element found at position %d",mid+1);

else

    printf("Element not found");

getch();

}

```

Output:

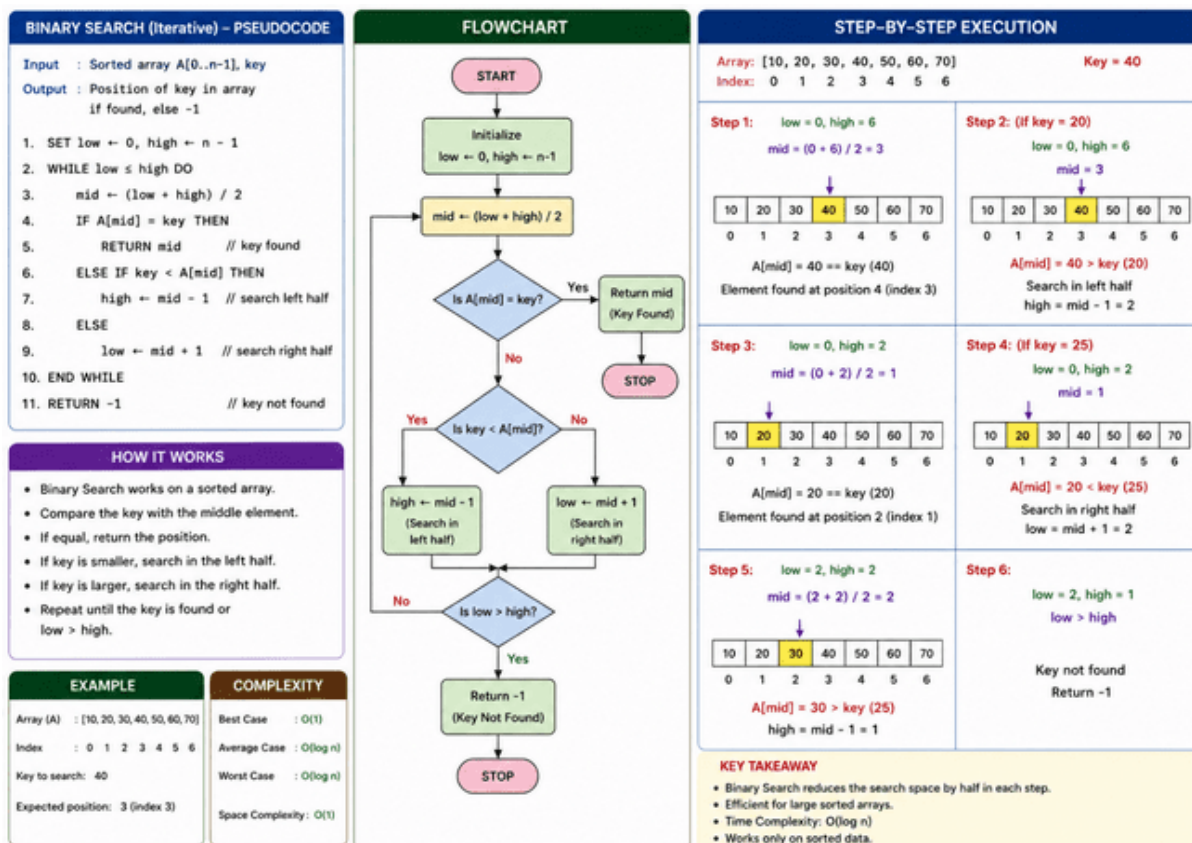
Enter number of elements: 5

Enter sorted elements:

10 20 30 40 50

Enter element to search: 30

Element found at position 3



2. Linear Search

```
#include<stdio.h>

#include<conio.h>

void main(){

    int a[100], n, i, key, found = 0;

    clrscr();

    printf("Enter number of elements: ");

    scanf("%d", &n);

    printf("Enter elements:\n");

    for(i=0; i<n; i++){

        scanf("%d", &a[i]);

    }

    printf("Enter element to search: ");

    scanf("%d", &key);

    for(i=0; i<n; i++){

        if(a[i] == key){

            found = 1;

            break;

        }

    }

    if(found)

        printf("Element found at position %d", i+1);

    else

        printf("Element not found");

    getch();

}
```

Output: Enter number of elements: 5

Enter elements:

10 25 30 45 50

Enter element to search: 45

Element found at position 4

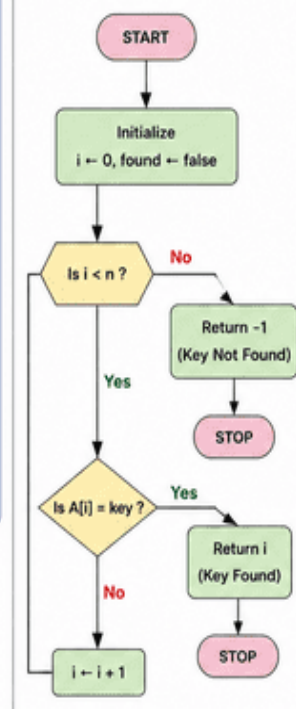
2. LINEAR SEARCH

PSEUDOCODE

Input : Array $A[0..n-1]$, key
Output : Position of key in array
if found, else -1

1. SET found $\leftarrow$ false
2. FOR $i \leftarrow 0$ TO $n-1$ DO
3. IF $A[i] = \text{key}$ THEN
4. RETURN i // key found
5. END IF
6. END FOR
7. RETURN -1 // key not found

FLOWCHART



SAMPLE OUTPUT

Enter number of elements: 5
Enter elements:
10 25 30 45 50
Enter element to search: 45
Element found at position 4

HOW IT WORKS (DRY RUN)

Array: [10, 25, 30, 45, 50]

Key = 45

Index: 0 1 2 3 4

Step 1: $i = 0$

10	25	30	45	50
0	1	2	3	4

Compare $A[0] = 10$ with key (45)

$10 \neq 45$

Not found. Move to next element.

Step 2: $i = 1$

10	25	30	45	50
0	1	2	3	4

Compare $A[1] = 25$ with key (45)

$25 \neq 45$

Not found. Move to next element.

Step 3: $i = 2$

10	25	30	45	50
0	1	2	3	4

Compare $A[2] = 30$ with key (45)

$30 \neq 45$

Not found. Move to next element.

Step 4: $i = 3$

10	20	30	45	50
0	1	2	3	4

Compare $A[3] = 45$ with key (45)

$45 = 45$

Key found at index 3.

Step 5: Search stops and returns index 3.
(Position = 3 (index 3))

3. Bubble Sort

```
#include<stdio.h>

#include<conio.h>

void main(){

    int a[100], n, i, j, temp;

    clrscr();

    printf("Enter number of elements: ");

    scanf("%d", &n);

    printf("Enter elements:\n");

    for(i=0; i<n; i++)

        scanf("%d", &a[i]);

    for(i=0; i<n-1; i++){

        for(j=0; j<n-i-1; j++){

            if(a[j] > a[j+1]){

                temp = a[j];

                a[j] = a[j+1];

                a[j+1] = temp;

            }

        }

    }

    printf("Sorted elements:\n");

    for(i=0; i<n; i++)

        printf("%d ", a[i]);

    getch();

}
```

Output: Enter number of elements: 5

Enter elements:

50 20 40 10 30

Sorted elements:

10 20 30 40 50

3. BUBBLE SORT

PSEUDOCODE

Input : Array A[0..n-1]
Output : Sorted array A

- FOR $i \leftarrow 0$ TO $n-2$
- FOR $j \leftarrow 0$ TO $n-i-2$
- IF $A[j] > A[j+1]$ THEN
- SWAP $A[j]$ and $A[j+1]$
- END FOR
- END FOR

HOW IT WORKS

- Compare adjacent elements.
- If the left element is greater, swap them.
- After each pass, the largest element moves to the end.
- Repeat until no more swaps are needed.

SAMPLE OUTPUT

Enter number of elements: 5
Enter elements:
50 20 40 10 30
Sorted elements:
10 20 30 40 50

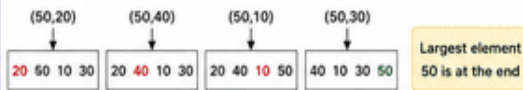
FLOWCHART



STEP-BY-STEP EXECUTION (DRY RUN)

Initial Array : 50 20 40 10 30
Index : 0 1 2 3 4

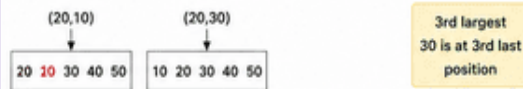
Pass 1 (i = 0)



Pass 2 (i = 1)



Pass 3 (i = 2)



Pass 4 (i = 3)



Array is sorted.

TIME AND SPACE COMPLEXITY

Best Case : $O(n)$ (when array is already sorted)
Average Case : $O(n^2)$
Worst Case : $O(n^2)$
Space Complexity: $O(1)$

4. Selection Sort

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main(){
```

```
    int a[100], n, i, j, min, temp;
```

```
    clrscr();
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter elements:\n");
```

```
    for(i=0; i<n; i++)
```

```
        scanf("%d", &a[i]);
```

```
    for(i=0; i<n-1; i++){
```

```
    min = i;
    for(j=i+1; j<n; j++){
        if(a[j] < a[min])
            min = j;
    }
    temp = a[i];
    a[i] = a[min];
    a[min] = temp;
}
printf("Sorted elements:\n");
for(i=0; i<n; i++)
    printf("%d ", a[i]);
getch();
}
```

Output: Enter number of elements: 5

Enter elements:

64 25 12 22 11

Sorted elements:

11 12 22 25 64

4. SELECTION SORT

PSEUDOCODE

Input : Array A[0..n-1]
Output : Sorted array A

- FOR $i \leftarrow 0$ TO $n-2$
- $\text{min} \leftarrow i$
- FOR $j \leftarrow i+1$ TO $n-1$
- IF $A[j] < A[\text{min}]$ THEN
- $\text{min} \leftarrow j$
- IF $\text{min} \neq i$ THEN
- SWAP $A[i]$ and $A[\text{min}]$
- END FOR

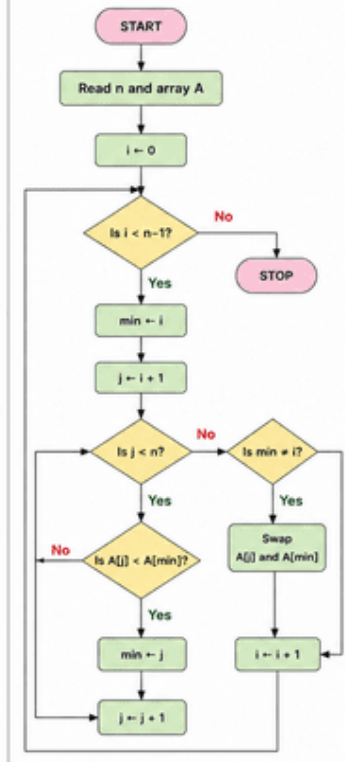
HOW IT WORKS

- Find the smallest element in the unsorted part.
- Swap it with the first unsorted element.
- Move the boundary of unsorted array one position forward.
- Repeat until the entire array is sorted.

SAMPLE OUTPUT

Enter number of elements: 5
Enter elements:
64 25 12 22 11
Sorted elements:
11 12 22 25 64

FLOWCHART



STEP-BY-STEP EXECUTION (DRY RUN)

Initial Array : 64 25 12 22 11
Index : 0 1 2 3 4

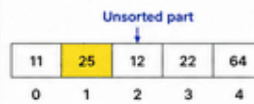
$i = 0$



Smallest in unsorted part is 11
(at index 4)
Swap $A[0]$ and $A[4]$

Array after swap:
11 25 12 22 64

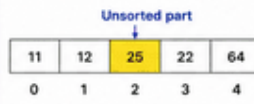
$i = 1$



Smallest in unsorted part is 12
(at index 2)
Swap $A[1]$ and $A[2]$

Array after swap:
11 12 25 22 64

$i = 2$



Smallest in unsorted part is 22
(at index 3)
Swap $A[2]$ and $A[3]$

Array after swap:
11 12 22 25 64

$i = 3$



Smallest in unsorted part is 25
(at index 3)
Already in correct position
No swap needed

Array is sorted.

TIME AND SPACE COMPLEXITY

Best Case : $O(n^2)$ (when array is already sorted)
Average Case : $O(n^2)$
Worst Case : $O(n^2)$
Space Complexity : $O(1)$

5. Insertion Sort, Iterative

```

#include<stdio.h>
#include<conio.h>
void main(){
    int a[100], n, i, j, key;
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    for(i=1;i<n;i++){
        key=a[i];
        j=i-1;
        while(j>=0 && a[j]>key){
            a[j+1]=a[j];
            j--;
        }
        a[j+1]=key;
    }
    printf("Sorted elements:\n");
  
```



```

for(i=0;i<n;i++)
    printf("%d ",a[i]);
getch();
}

```

Output: Enter number of elements: 5

Enter elements:

12 11 13 5 6

Sorted elements:

5 6 11 12 13

5. INSERTION SORT (ITERATIVE)

PSEUDOCODE

Input : Array A[0..n-1]
Output : Sorted array A

- FOR $i \leftarrow 1$ TO $n-1$
- key $\leftarrow A[i]$
- $j \leftarrow i - 1$
- WHILE $j \geq 0$ AND $A[j] > \text{key}$
- $A[j + 1] \leftarrow A[j]$
- $j \leftarrow j - 1$
- END WHILE
- $A[j + 1] \leftarrow \text{key}$
- END FOR

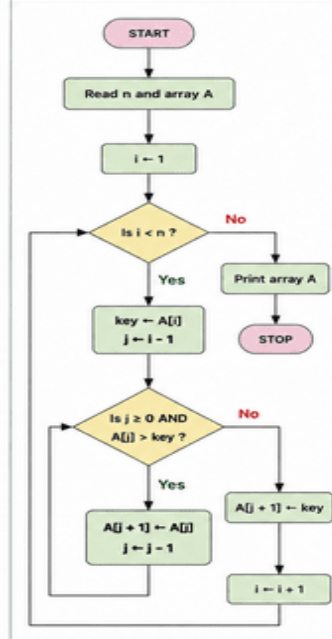
HOW IT WORKS

- Start from the second element.
- Compare the key (current element) with elements in the sorted left part.
- Shift greater elements one position ahead.
- Insert the key in its correct position.
- Repeat for all elements.

SAMPLE OUTPUT

Enter number of elements: 5
Enter elements:
12 11 13 5 6
Sorted elements:
5 6 11 12 13

FLOWCHART



STEP-BY-STEP EXECUTION (DRY RUN)

Initial Array : 12 11 13 5 6
Index : 0 1 2 3 4

Step 1: $i = 1$, key = 11

Compare 11 with 12 $\rightarrow 11 < 12$
Shift 12 to right
Insert 11 at position 0

11	12	13	5	6
0	1	2	3	4

Step 2: $i = 2$, key = 13

Compare 13 with 12 $\rightarrow 13 > 12$
No shift needed
Insert 13 at position 2

11	12	13	5	6
0	1	2	3	4

Step 3: $i = 3$, key = 5

Compare 5 with 13 $\rightarrow 5 < 13$ (shift)
Compare 5 with 12 $\rightarrow 5 < 12$ (shift)
Compare 5 with 11 $\rightarrow 5 < 11$ (shift)
Insert 5 at position 0

5	11	12	13	6
0	1	2	3	4

Step 4: $i = 4$, key = 6

Compare 6 with 13 $\rightarrow 6 < 13$ (shift)
Compare 6 with 12 $\rightarrow 6 < 12$ (shift)
Compare 6 with 11 $\rightarrow 6 < 11$ (shift)
Compare 6 with 5 $\rightarrow 6 > 5$ (stop)
Insert 6 at position 1

5	6	11	12	13
0	1	2	3	4

Array is sorted.

TIME AND SPACE COMPLEXITY

Best Case : $O(n)$ (array already sorted)
Average Case : $O(n^2)$
Worst Case : $O(n^2)$ (array in reverse order)
Space Complexity : $O(1)$ (in-place sorting)

6. Shell Sort, Iterative

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main(){
```

```
    int a[100], n, i, j, gap, temp;
```

```
    clrscr();
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter elements:\n");
```

```
    for(i=0;i<n;i++)
```

```
        scanf("%d",&a[i]);
```

```
    for(gap=n/2; gap>0; gap=gap/2){
```

```
        for(i=gap; i<n; i++){
```

```
            temp=a[i];
```

```
            for(j=i; j>=gap && a[j-gap]>temp; j=j-gap){
```

```
                a[j]=a[j-gap];
```

```

    }
    a[j]=temp;
}
}
printf("Sorted elements:\n");
for(i=0;i<n;i++)
    printf("%d ",a[i]);
getch();
}

```

Output

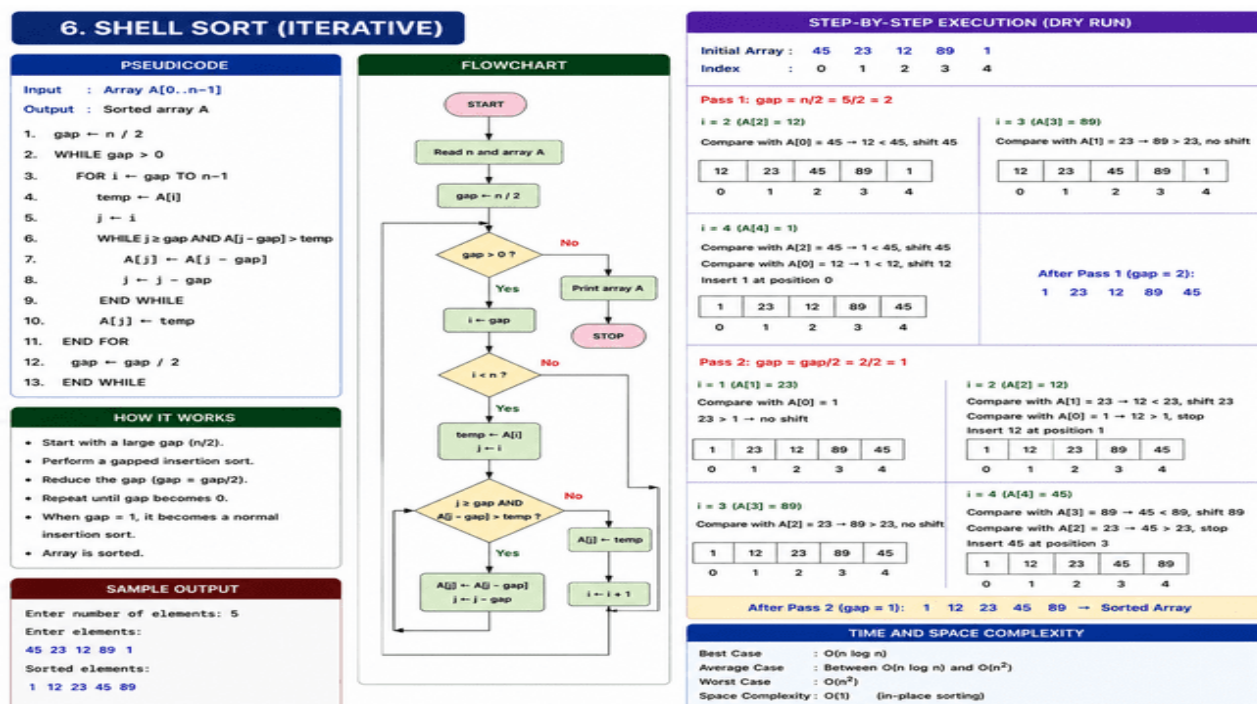
Enter number of elements: 5

Enter elements:

45 23 12 89 1

Sorted elements:

1 12 23 45 89



7. Merge Sort

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int a[50];
```

```
void merge(int low,int mid,int high){
```

```
    int h,i,j,b[50],k;
```

```
    h=low;
```

```

i=low;
j=mid+1;
while((h<=mid)&&(j<=high)){
    if(a[h]<=a[j]){
        b[i]=a[h];
        h++;
    }
    else{
        b[i]=a[j];
        j++;
    }
    i++;
}
if(h>mid){
    for(k=j;k<=high;k++){
        b[i]=a[k];
        i++;
    }
}
else{
    for(k=h;k<=mid;k++){
        b[i]=a[k];
        i++;
    }
}
for(k=low;k<=high;k++)
    a[k]=b[k];
}

void mergesort(int low,int high){
    int mid;
    if(low<high){

```

```

        mid=(low+high)/2;
        mergesort(low,mid);
        mergesort(mid+1,high);
        merge(low,mid,high);
    }
}

void main(){
    int n,i;
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    mergesort(0,n-1);
    printf("Sorted elements:\n");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
    getch();
}

```

Output: Enter number of elements: 5

Enter elements:

50 20 40 10 30

Sorted elements:

10 20 30 40 50

7. MERGE SORT

PSEUDOCODE

Input : Array A[0..n-1]
Output : Sorted array A

MERGE-SORT(A, low, high)

1. if low < high
2. mid ← (low + high) / 2
3. MERGE-SORT(A, low, mid)
4. MERGE-SORT(A, mid+1, high)
5. MERGE(A, low, mid, high)

MERGE(A, low, mid, high)

1. i ← low, j ← mid+1, k ← low
2. while i ≤ mid and j ≤ high
3. if A[i] ≤ A[j]
4. temp[k] ← A[i]; i ← i + 1
5. else
6. temp[k] ← A[j]; j ← j + 1
7. k ← k + 1
8. while i ≤ mid
9. temp[k] ← A[i]; i ← i + 1; k ← k + 1
10. while j ≤ high
11. temp[k] ← A[j]; j ← j + 1; k ← k + 1
12. for t ← low to high
13. A[t] ← temp[t]

HOW IT WORKS

- Divide the array into two halves.
- Recursively sort both halves.
- Merge the two sorted halves.
- Continue until the whole array is sorted.

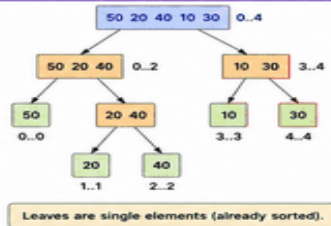
SAMPLE OUTPUT

Enter number of elements: 5
Enter elements:
50 20 40 10 30
Sorted elements:
10 20 30 40 50

FLOWCHART

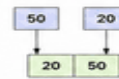


RECURSION TREE (DIVIDE PHASE)



MERGE PHASE (STEP-BY-STEP)

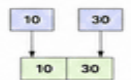
Step 1: Merge [50] and [20]



Step 2: Merge [20 50] and [40]



Step 3: Merge [10] and [30]



Step 4: Merge [20 40 50] and [10 30]



Step 4 Detailed Merge:



- 1) Compare 20 and 10 → 10 is smaller → add 10
Left Array: [20, 40, 50] Right Array: [10, 30] Merged Array: [10, , , ,]
i → 20 j → 30
- 2) Compare 20 and 30 → 20 is smaller → add 20
Left Array: [20, 40, 50] Right Array: [10, 30] Merged Array: [10, 20, , ,]
i → 40 j → 30
- 3) Compare 40 and 30 → 30 is smaller → add 30
Left Array: [40, 50] Right Array: [10, 20, 30] Merged Array: [10, 20, 30, ,]
i → 40 j → 30
- 4) Right array finished → copy remaining left
Left Array: [40, 50] Right Array: [, 20, 30, 40, 50] Merged Array: [, 20, 30, 40, 50]
k → 5 (done)

Final Sorted Array: 10 20 30 40 50

TIME AND SPACE COMPLEXITY

Best Case : $O(n \log n)$
Average Case : $O(n \log n)$
Worst Case : $O(n \log n)$
Space Complexity : $O(n)$ (extra space for temp array)

8. Quick Sort

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int partition(int a[],int low,int high){
```

```
    int pivot,i,j,temp;
```

```
    pivot=a[low];
```

```
    i=low+1;
```

```
    j=high;
```

```
    while(i<=j) {
```

```
        while(a[i]<=pivot && i<=high)
```

```
            i++;
```

```
        while(a[j]>pivot)
```

```
            j--;
```

```
        if(i<j){
```

```
            temp=a[i];
```

```
            a[i]=a[j];
```

```
            a[j]=temp;
```

```
        }
```

```

    }
    temp=a[low];
    a[low]=a[j];
    a[j]=temp;
    return j;
}

void quicksort(int a[],int low,int high){
    int p;
    if(low<high){
        p=partition(a,low,high);
        quicksort(a,low,p-1);
        quicksort(a,p+1,high);
    }
}

void main(){
    int a[20],n,i;
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    quicksort(a,0,n-1);
    printf("Sorted elements:\n");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
    getch();
}

```

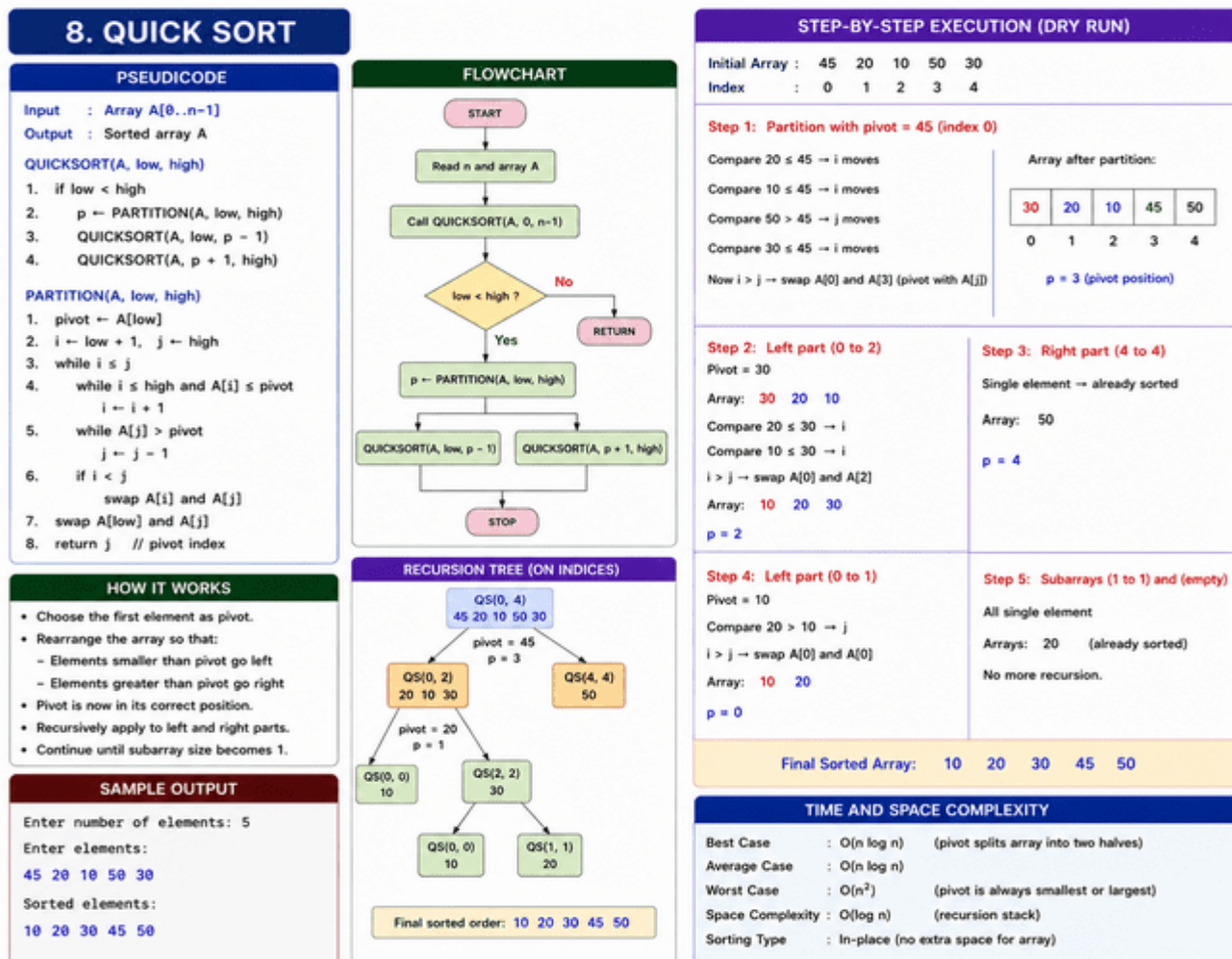
Enter number of elements: 5

Enter elements:

45 20 10 50 30

Sorted elements:

10 20 30 45 50



```

        largest=right;
    if(largest!=i){
        temp=a[i];
        a[i]=a[largest];
        a[largest]=temp;
        heapify(a,n,largest);
    }
}

void heapsort(int a[], int n){
    int i, temp;
    for(i=n/2-1;i>=0;i--){
        heapify(a,n,i);
    }
    for(i=n-1;i>=0;i--){
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;

        heapify(a,i,0);
    }
}

void main(){
    int a[20],n,i;

    clrscr();

    printf("Enter number of elements: ");
    scanf("%d",&n);

    printf("Enter elements:\n");
    for(i=0;i<n;i++){
        scanf("%d",&a[i]);
    }

    heapsort(a,n);

    printf("Sorted elements:\n");
    for(i=0;i<n;i++){

```



```

printf("%d ",a[i]);

getch();

}

```

Enter number of elements: 5

Enter elements:

40 10 30 50 20

Sorted elements:

10 20 30 40 50

9. HEAP SORT

PSEUDOCODE

```

Input  : Array A[0..n-1]
Output : Sorted array A

HEAPSORT(A, n)
1. Build Max Heap from A[0..n-1]
2. FOR i ← n-1 DOWNTO 1
3.   swap A[0] and A[i]
4.   heapify(A, i, 0)
5. END FOR

BUILD-MAX-HEAP(A, n)
1. FOR i ← n/2 - 1 DOWNTO 0
2.   heapify(A, n, i)
3. END FOR

HEAPIFY(A, n, i)
1. largest ← i
2. l ← 2*i + 1 // left child
3. r ← 2*i + 2 // right child
4. IF l < n AND A[l] > A[largest]
5.   largest ← l
6. IF r < n AND A[r] > A[largest]
7.   largest ← r
8. IF largest ≠ i
9.   swap A[i] and A[largest]
10.  heapify(A, n, largest)

```

HOW IT WORKS

- Build a max heap from the array.
- The largest element will be at root (A[0]).
- Swap the root with last element.
- Reduce heap size by 1.
- Heapify the root.
- Repeat until heap size becomes 1.

SAMPLE OUTPUT

```

Enter number of elements: 5
Enter elements:
40 10 30 50 20
Sorted elements:
10 20 30 40 50

```

FLOWCHART

BUILD MAX HEAP (Initial Array)

Array: 40 10 30 50 20
Index: 0 1 2 3 4

$i = n/2 - 1 = 5/2 - 1 = 1$

1) $i = 1$, heapify(1)

2) $i = 0$, heapify(0)

Max Heap: 50 40 30 10 20
Index: 0 1 2 3 4

STEP-BY-STEP EXECUTION (DRY RUN)

Initial Array : 40 10 30 50 20
Max Heap : 50 40 30 10 20

Step	Operation	Heap (Unsorted Part)	Sorted Part
1	Swap A[0] and A[4] (heap size = 4) heapify(0)	2040301050 01234 4020301050 01234	50
2	Swap A[0] and A[3] (heap size = 3) heapify(0)	1020304050 01234 3020104050 01234	40 50
3	Swap A[0] and A[2] (heap size = 2) heapify(0)	1020304050 01234 2010304050 01234	30 40 50
4	Swap A[0] and A[1] (heap size = 1) heapify(0)	1020304050 01234 1020304050 01234	20 30 40 50
Final Sorted Array: 10 20 30 40 50			

TIME AND SPACE COMPLEXITY

Build Max Heap	: $O(n)$
Heapify (per call)	: $O(\log n)$
Total Time	: $O(n \log n)$
Best Case	: $O(n \log n)$
Average Case	: $O(n \log n)$
Worst Case	: $O(n \log n)$
Space Complexity	: $O(1)$ (In-place sorting)

10. Dijkstra Algorithm

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#define MAX 10
```

```
void main(){
```

```
    int cost[MAX][MAX],dist[MAX],visited[MAX];
```

```

int i,j,n, start,nextnode,mindist;

clrscr();

printf("Enter number of vertices: ");

scanf("%d",&n);

printf("Enter cost matrix:\n");

for(i=0;i<n;i++){

    for(j=0;j<n;j++){

        scanf("%d",&cost[i][j]);

        if(cost[i][j]==0)

            cost[i][j]=INF;

    }

}

printf("Enter starting node: ");

scanf("%d",&start);

for(i=0;i<n;i++){

    dist[i]=cost[start][i];

    visited[i]=0;

}

dist[start]=0;

visited[start]=1;

for(i=1;i<n-1;i++){

    mindist=INF

    for(j=0;j<n;j++){

        if(dist[j]<mindist && !visited[j]){

mindist=dist[j];

            nextnode=j;

        }

    }

    visited[nextnode]=1;

    for(j=0;j<n;j++){

        if(!visited[j]){

```

```

        if(mindist+cost[nextnode][j]<dist[j]) {
            dist[j]=mindist+cost[nextnode][j];
        }
    }
}

printf("Shortest distances:\n");
for(i=0;i<n;i++){
    printf("%d -> %d = %d\n",start,i,dist[i]);
}

getch();
}

```

Enter number of vertices: 4

Enter cost matrix:

0 2 4 0

2 0 1 7

4 1 0 3

0 7 3 0

Enter starting node: 0

Shortest distances:

0 -> 0 = 0

0 -> 1 = 2

0 -> 2 = 3

0 -> 3 = 6

10. DIJKSTRA ALGORITHM

PSEUDOCODE

Input : Graph $G(V, E)$ represented by cost matrix $\text{cost}[0..n-1][0..n-1]$, source vertex s
Output : Shortest distances $\text{dist}[]$ from s to all vertices

DIJKSTRA(cost, n, s)

- for $i \leftarrow 0$ to $n-1$ do
 $\text{dist}[i] \leftarrow \text{cost}[s][i]$
 $\text{visited}[i] \leftarrow \text{FALSE}$
- $\text{dist}[s] \leftarrow 0$
- $\text{visited}[s] \leftarrow \text{TRUE}$
- for $\text{count} \leftarrow 1$ to $n-2$ do
 $\text{mindist} \leftarrow \text{INF}$
 $\text{nextnode} \leftarrow -1$
 for $v \leftarrow 0$ to $n-1$ do
 if ($\text{visited}[v] = \text{FALSE}$ and $\text{dist}[v] < \text{mindist}$) then
 $\text{mindist} \leftarrow \text{dist}[v]$
 $\text{nextnode} \leftarrow v$
 $\text{visited}[\text{nextnode}] \leftarrow \text{TRUE}$
 for $v \leftarrow 0$ to $n-1$ do
 if ($\text{visited}[v] = \text{FALSE}$) then
 if ($\text{mindist} + \text{cost}[\text{nextnode}][v] < \text{dist}[v]$) then
 $\text{dist}[v] \leftarrow \text{mindist} + \text{cost}[\text{nextnode}][v]$
- return $\text{dist}[]$

(INF is a very large value)

HOW IT WORKS

- Initialize distances from source to all vertices.
- Repeatedly pick the unvisited vertex with minimum distance.
- Update the distances of its unvisited neighbors.
- Continue until all vertices are visited.

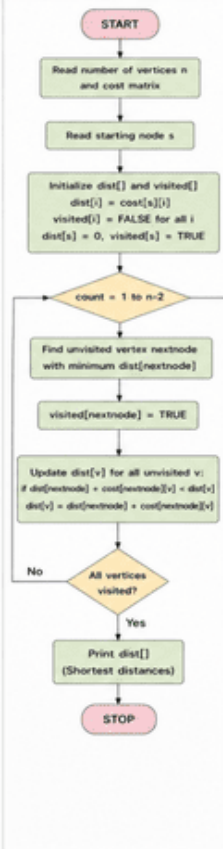
COMPLEXITY

- Time Complexity : $O(n^2)$ (using cost matrix)
- Space Complexity : $O(n)$
- Works only for graphs with non-negative edge weights.

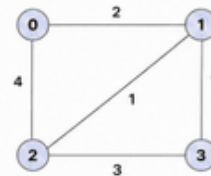
SAMPLE INPUT

Enter number of vertices: 4
 Enter cost matrix:
 0 2 4 0
 2 0 1 7
 4 1 0 3
 0 7 3 0
 Enter starting node: 0

FLOWCHART



EXAMPLE GRAPH (UNDIRECTED WEIGHTED)



INPUT

Number of vertices: 4
 Cost Matrix:
 0 2 4 0
 2 0 1 7
 4 1 0 3
 0 7 3 0
 Starting node: 0

OUTPUT

Shortest distances:
 0 -> 0 = 0
 0 -> 1 = 2
 0 -> 2 = 3
 0 -> 3 = 6

STEP-BY-STEP EXECUTION

Step	Visited Set	Selected Vertex	Distance Array (dist[])	Explanation
0	{0}	-	0 2 4 INF	Initialize from source 0
1	{0}	1	0 2 3 9	Pick 1 (min=2), update 2 and 3
2	{0, 1}	2	0 2 3 6	Pick 2 (min=3), update 3
3	{0, 1, 2}	3	0 2 3 6	Pick 3 (min=6), no update

DRY RUN (DETAILS)

Step	Current Vertex	Visited[]	dist[]	Action/Update
0	-	1 0 0 0	0 2 4 INF	Initialization
1	1	1 1 0 0	0 2 3 9	Update 2 = 3, 3 = 9
2	2	1 1 1 0	0 2 3 6	Update 3 = 6
3	3	1 1 1 1	0 2 3 6	No update

FINAL RESULT

Shortest distances from source 0 : [0, 2, 3, 6]

11. Kruskal Algorithm

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int parent[10];
```

```
int find(int i){
```

```
    while(parent[i])
```

```
        i=parent[i];
```

```
    return i;
```

```
}
```

```
int uni(int i,int j){
```

```
    if(i!=j){
```

```
        parent[j]=i;
```

```
        return 1;
```

```
    }
```

```
    return 0;
```

```
}
```

```

void main(){

    int cost[10][10],i,j,n,min,a,b,u,v,ne=1,mincost=0;

    clrscr();

    printf("Enter number of vertices: ");

    scanf("%d",&n);

    printf("Enter cost matrix:\n");

    for(i=1;i<=n;i++){

        for(j=1;j<=n;j++){

            scanf("%d",&cost[i][j]);

            if(cost[i][j]==0)

                cost[i][j]=999;

        }

    }

    while(ne<n){

        min=999;

        for(i=1;i<=n;i++){

            for(j=1;j<=n;j++){

                if(cost[i][j]<min){

                    min=cost[i][j];

                    a=u=i;

                    b=v=j;

                }

            }

        }

        u=find(u);

        v=find(v);

        if(uni(u,v))

        {

            printf("\nEdge %d:(%d,%d) cost=%d",ne++,a,b,min);

            mincost+=min;

        }

    }

```

```
        cost[a][b]=cost[b][a]=999;
    }
    printf("\nMinimum cost = %d",mincost);
    getch();
}
```

Enter number of vertices: 4

Enter cost matrix:

0 2 0 6

2 0 3 8

0 3 0 0

6 8 0 0

Edge 1:(1,2) cost=2

Edge 2:(2,3) cost=3

Edge 3:(1,4) cost=6

Minimum cost = 11

11. KRUSKAL ALGORITHM

PSEUDOCODE

Input : Graph $G(V, E)$ represented by cost matrix
Output : Minimum Spanning Tree (MST)

KRUSKAL(G)

1. Create a set T to store the MST (initially empty)
2. For each vertex v in V
 Make-Set(v)
3. Create a list of all edges from the cost matrix
4. Sort all edges in non-decreasing order of cost
5. For each edge (u, v) in sorted order
 If **Find-Set**(u) $\neq$ **Find-Set**(v) // u and v are in different sets (no cycle will form)
 Add (u, v) to T
 Union(**Find-Set**(u), **Find-Set**(v))
 If $|T| = |V| - 1$ // MST completed
 Break
6. Return T

Make-Set(v)

1. **parent**[v] $\leftarrow v$
2. **rank**[v] $\leftarrow 0$

Find-Set(v)

1. If **parent**[v] $\neq v$
 parent[v] $\leftarrow$ **Find-Set**(**parent**[v]) // Path compression
2. Return **parent**[v]

Union(x, y)

1. If **rank**[x] < **rank**[y]
 parent[x] $\leftarrow y$
3. Else if **rank**[x] > **rank**[y]
 parent[y] $\leftarrow x$
5. Else
 parent[y] $\leftarrow x$
 rank[x] $\leftarrow$ **rank**[x] + 1

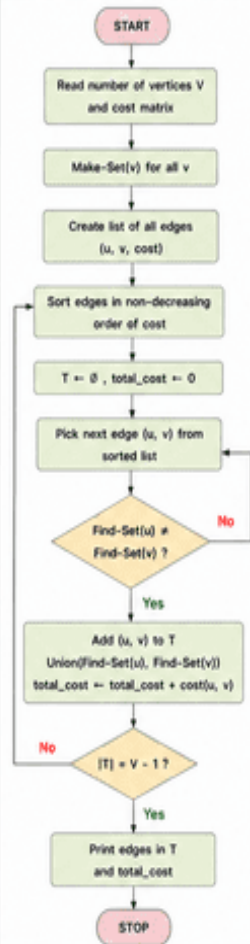
HOW IT WORKS

- Consider edges in increasing order of weight.
- Add an edge if it does not form a cycle.
- Use Union-Find (Disjoint Set) to detect cycles.
- Stop when MST has $(V-1)$ edges.

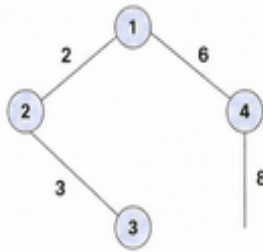
COMPLEXITY

- Sorting edges : $O(E \log E)$
 - Union-Find operations : $O(E \alpha(V))$
 - Overall Time Complexity : $O(E \log E)$
 - Space Complexity : $O(V)$
- Where $\alpha(V)$ is the inverse Ackermann function (very small in practice).

FLOWCHART



EXAMPLE GRAPH (UNDIRECTED WEIGHTED)



INPUT

Number of vertices: 4
 Cost Matrix:

0	2	0	6
2	0	3	8
0	3	0	0
6	8	0	0

ALL EDGES (SORTED BY WEIGHT)

Edge (u, v)	Cost
(1, 2)	2
(2, 3)	3
(1, 4)	6
(2, 4)	8

OUTPUT

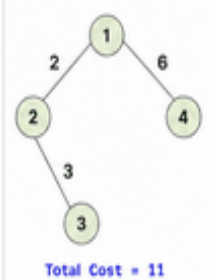
Edge 1:(1,2) cost=2
 Edge 2:(2,3) cost=3
 Edge 3:(1,4) cost=6
 Minimum cost = 11

STEP-BY-STEP EXECUTION

Step	Edge Considered (u, v)	Cost	Root of u	Root of v	Action	Edges in MST (T)	Total Cost
0	-	-	-	-	Start	{}	0
1	(1, 2)	2	1	2	Add	{(1,2)}	2
2	(2, 3)	3	1	3	Add	{(1,2), (2,3)}	5
3	(1, 4)	6	1	4	Add	{(1,2), (2,3), (1,4)}	11
4	(2, 4)	8	1	1	Skip (cycle)	{(1,2), (2,3), (1,4)}	11

MST completed ($|T| = V-1 = 3$ edges)

RESULTING MST



SAMPLE INPUT

Enter number of vertices: 4
 Enter cost matrix:

0	2	0	6
2	0	3	8
0	3	0	0
6	8	0	0

SAMPLE OUTPUT

Edge 1:(1,2) cost=2
 Edge 2:(2,3) cost=3
 Edge 3:(1,4) cost=6
 Minimum cost = 11

12. Prim Algorithm

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main(){
```

```
    int cost[10][10],visited[10]={0};
```

```
    int i,j,n,ne=1,min,mincost=0,a,b;
```

```
    clrscr();
```

```
    printf("Enter number of vertices: ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter cost matrix:\n");
```

```
    for(i=1;i<=n;i++){
```

```

for(j=1;j<=n;j++){
    scanf("%d",&cost[i][j]);
    if(cost[i][j]==0)
        cost[i][j]=999;
}
visited[1]=1;
while(ne<n){
    min=999;
    for(i=1;i<=n;i++){
        for(j=1;j<=n;j++){
            if(cost[i][j]<min){
                if(visited[i]!=0){
                    min=cost[i][j];
                    a=i;
                    b=j;
                }
            }
        }
    }
    if(visited[b]==0){
        printf("\nEdge %d:(%d %d) cost=%d",ne++,a,b,min);
        mincost+=min;
        visited[b]=1;
    }
    cost[a][b]=cost[b][a]=999;
}
printf("\nMinimum cost=%d",mincost);
getch();
}

```

Enter number of vertices: 4

Enter cost matrix:

0 2 0 6

2 0 3 8

0 3 0 0

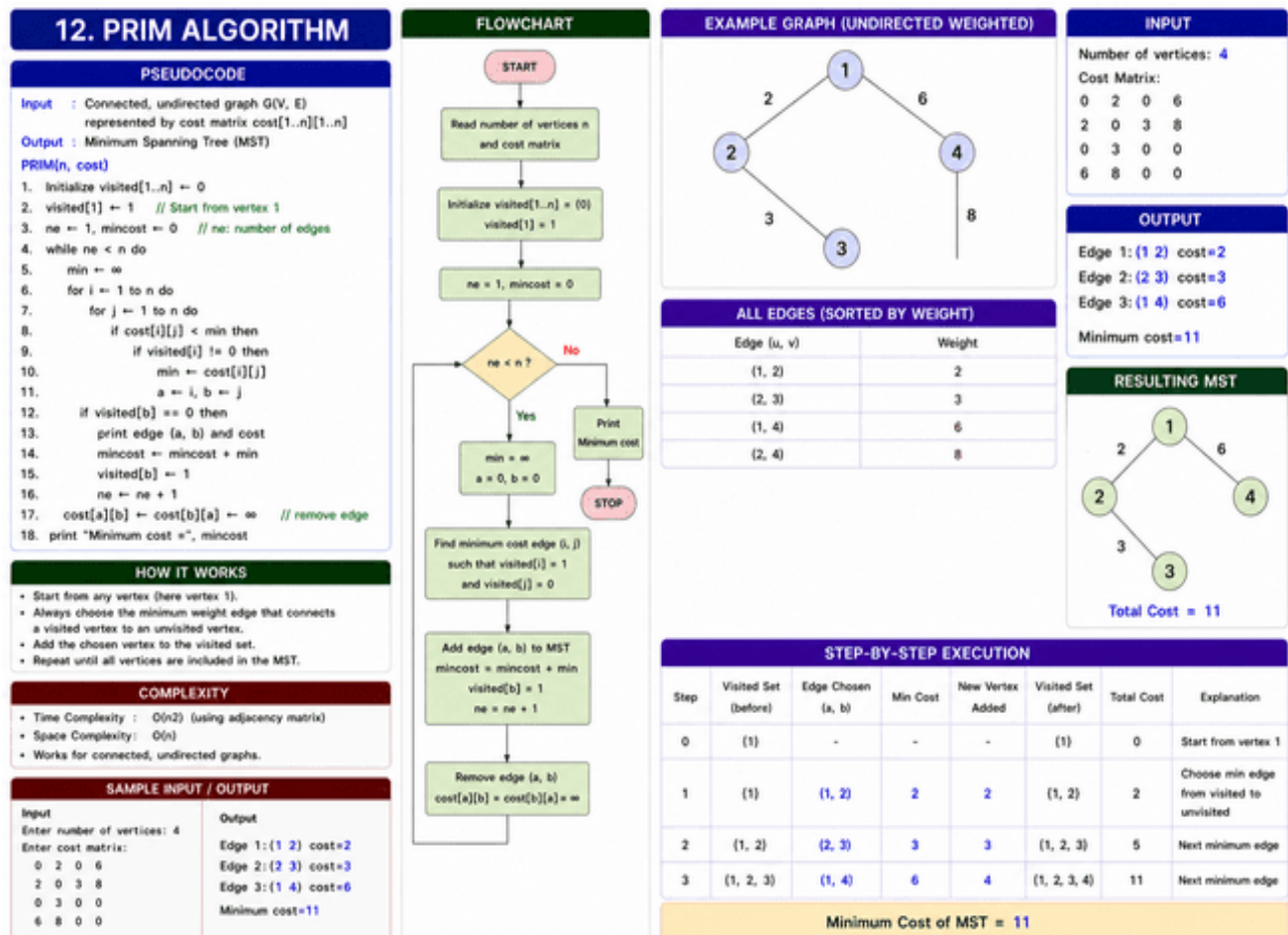
6 8 0 0

Edge 1:(1 2) cost=2

Edge 2:(2 3) cost=3

Edge 3:(1 4) cost=6

Minimum cost=11



13. fractional knapsack

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main(){
```

```
    int i, j, n, temp;
```

```
int weight[20], profit[20], capacity;

float ratio[20], total = 0;

clrscr();

printf("Enter number of items: ");

scanf("%d", &n);

printf("Enter profits and weights:\n");

for(i=0; i<n; i++){

    scanf("%d%d", &profit[i], &weight[i]);

    ratio[i] = (float)profit[i] / weight[i];

}

printf("Enter capacity: ");

scanf("%d", &capacity);

for(i=0; i<n; i++){

    for(j=i+1; j<n; j++){

        if(ratio[i] < ratio[j]){

            temp = profit[i];

            profit[i] = profit[j];

            profit[j] = temp;

            temp = weight[i];

            weight[i] = weight[j];

            weight[j] = temp;

            ratio[i] = (float)profit[i] / weight[i];

            ratio[j] = (float)profit[j] / weight[j];

        }

    }

}

for(i=0; i<n; i++){

    if(weight[i] <= capacity){

        total += profit[i];

        capacity -= weight[i];

    }

}
```

```
    else
    {
        total += ratio[i] * capacity;
        break;
    }
}

printf("Maximum Profit = %.2f", total);

getch();
}
```

Output: Enter number of items: 3

Enter profits and weights:

60 10

100 20

120 30

Enter capacity: 50

Maximum Profit = 240.00

13. FRACTIONAL KNAPSACK

PSEUDOCODE

Input : n = number of items
 $profit[0..n-1]$, $weight[0..n-1]$, capacity

Output : Maximum profit

FRACTIONAL-KNAPSACK($profit, weight, n, capacity$)

- for $i \leftarrow 0$ to $n - 1$ do
- $ratio[i] \leftarrow profit[i] / weight[i]$
- Sort items in decreasing order of $ratio[i]$
- $total \leftarrow 0$
- for $i \leftarrow 0$ to $n - 1$ do
- if $weight[i] \leq capacity$ then
- $total \leftarrow total + profit[i]$
- $capacity \leftarrow capacity - weight[i]$
- else
- $total \leftarrow total + ratio[i] * capacity$
- break
- return total

HOW IT WORKS

- Calculate profit/weight ratio for each item.
- Sort items in decreasing order of ratio.
- Add items to knapsack one by one.
- If the whole item can be added, add it completely.
- If not, add fraction of the item to fill remaining capacity.
- Stop when capacity becomes 0.

COMPLEXITY

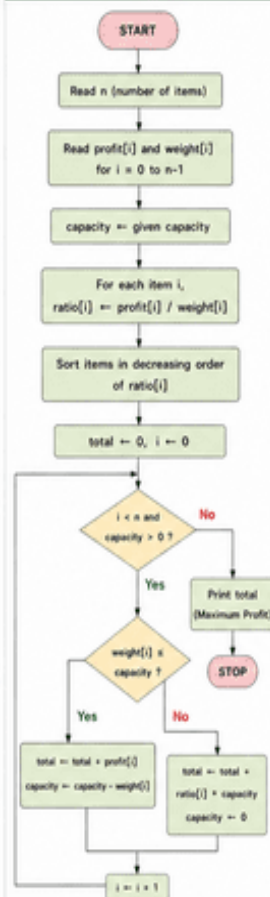
- Time Complexity : $O(n \log n)$ (due to sorting)
- Space Complexity : $O(n)$
- Greedy Approach : Yes

SAMPLE INPUT / OUTPUT

Input
 Enter number of items: 3
 Enter profits and weights:
 60 10
 100 20
 120 30
 Enter capacity: 50

Output
 Maximum Profit = 240.00

FLOWCHART



EXAMPLE

Knapsack Capacity = 50

Item	Profit	Weight	Profit/Weight Ratio
1	60	10	6.00
2	100	20	5.00
3	120	30	4.00

After sorting by decreasing ratio (Profit/Weight)

Order	Item	Profit	Weight	Ratio
1	1	60	10	6.00
2	2	100	20	5.00
3	3	120	30	4.00

STEP-BY-STEP EXECUTION

Step	Item Considered	Profit	Weight	Ratio	Capacity Before	Action	Capacity After	Total Profit
0	-	-	-	-	50	Start	50	0.00
1	Item 1	60	10	6.00	50	Whole item taken (10 ≤ 50)	40	60.00
2	Item 2	100	20	5.00	40	Whole item taken (20 ≤ 40)	20	160.00
3	Item 3	120	30	4.00	20	Take fraction (20 < 30) Fraction taken = 20/30 Profit added = 4 × 20 = 80	0	240.00

Maximum Profit = 240.00

VISUAL REPRESENTATION



14. 0/1 Knapsack

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
    int i, j, n, m;
```

```
    int w[10], p[10], k[10][10];
```

```
    clrscr();
```

```
    printf("Enter number of items: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter profits:\n");
```

```
    for(i=1; i<=n; i++)
```

```
        scanf("%d", &p[i]);
```

```
    printf("Enter weights:\n");
```

```
    for(i=1; i<=n; i++)
```

```

    scanf("%d", &w[i]);
printf("Enter capacity: ");
scanf("%d", &m);
for(i=0; i<=n; i++)
{
    for(j=0; j<=m; j++)
    {
        if(i==0 || j==0)
            k[i][j] = 0;
        else if(w[i] <= j)
        {
            if(p[i] + k[i-1][j-w[i]] > k[i-1][j])
                k[i][j] = p[i] + k[i-1][j-w[i]];
            else
                k[i][j] = k[i-1][j];
        }
        else
            k[i][j] = k[i-1][j];
    }
}
printf("Maximum Profit = %d", k[n][m]);
getch();
}

```

Output: Enter number of items: 3

Enter profits:

60 100 120

Enter weights:

10 20 30

Enter capacity: 50

Maximum Profit = 220

14. 0/1 KNAPSACK

PSEUDOCODE

Input : n = number of items
 $p[1..n]$ = profits of items
 $w[1..n]$ = weights of items
 m = capacity of knapsack

Output : Maximum profit

0-1-KNAPSACK(p, w, n, m)

```

1. Create a table  $k[0..n][0..m]$ 
2. for  $i = 0$  to  $n$  do
3.   for  $j = 0$  to  $m$  do
4.     if  $i = 0$  or  $j = 0$  then
5.        $k[i][j] = 0$ 
6.     else if  $w[i] \leq j$  then
7.       if  $p[i] + k[i-1][j-w[i]] > k[i-1][j]$  then
8.          $k[i][j] = p[i] + k[i-1][j-w[i]]$ 
9.       else
10.         $k[i][j] = k[i-1][j]$ 
11.      else
12.         $k[i][j] = k[i-1][j]$ 
13. return  $k[n][m]$ 
    
```

HOW IT WORKS

- Build a table $k[i][j]$ where $i = 0..n$ and $j = 0..m$.
- $k[i][j]$ stores the maximum profit using first i items with capacity j .
- If the current item's weight is greater than j , copy the value from above $k[i-1][j]$.
- Otherwise, choose the maximum between (include item) or (exclude item).
- The answer is stored in $k[n][m]$.

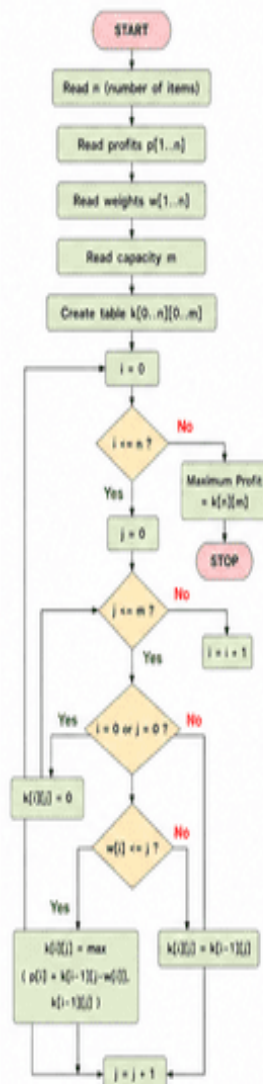
COMPLEXITY

- Time Complexity : $O(n \times m)$
- Space Complexity : $O(n \times m)$

SAMPLE INPUT / OUTPUT

Input	Output
Enter number of items: 3	Maximum Profit = 220
Enter profits: 60 100 120	
Enter weights: 10 20 30	
Enter capacity: 50	

FLOWCHART



ITEMS

Item (i)	Profit p[i]	Weight w[i]	Profit/Weight Ratio
1	60	10	6.00
2	100	20	5.00
3	120	30	4.00

INPUT

Number of items (n): 3
 Profits: 60 100 120
 Weights: 10 20 30
 Capacity (m): 50

DP TABLE $k[i][j]$ (Maximum Profit using first i items with capacity j)

i \ j	0	10	20	30	40	50
0	0	0	0	0	0	0
1	0	60	60	60	60	60
2	0	60	100	160	160	160
3	0	60	100	120	180	220

Rows: $i = 0..n$ (items considered) Columns: $j = 0..m$ (capacity)

HOW THE TABLE IS FILLED (Example Explanations)

i	j (Capacity)	Decision	Calculation	$k[i][j]$
1	10	$w[1] = 10 \leq 10$	$\max(p[1] + k[0][0], k[0][10]) = \max(60, 0)$	60
1	20	$w[1] = 10 \leq 20$	$\max(p[1] + k[0][10], k[0][20]) = \max(60, 0)$	60
2	20	$w[2] = 20 \leq 20$	$\max(p[2] + k[1][0], k[1][20]) = \max(100, 60)$	100
2	30	$w[2] = 20 \leq 30$	$\max(p[2] + k[1][10], k[1][30]) = \max(160, 60)$	160
3	50	$w[3] = 30 \leq 50$	$\max(p[3] + k[2][20], k[2][50]) = \max(220, 160)$	220

SELECTED ITEMS (Backtracking)

Start at $k[3][50] = 220$

- $k[3][50] \neq k[2][50] \Rightarrow$ Item 3 is included ($w=30$)
 New capacity = $50 - 30 = 20$
- $k[2][20] \neq k[1][20] \Rightarrow$ Item 2 is included ($w=20$)
 New capacity = $20 - 20 = 0$
- Capacity = 0 $\Rightarrow$ Stop

Items included in optimal solution

Item	Profit	Weight
2	100	20
3	120	30
Total Profit = 220		

Maximum Profit = 220

15. Floyd Warshall Algorithm

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int a[10][10], n;
```

```
int i, j, k;
```

```
clrscr();
```

```
printf("Enter number of vertices: ");
```

```

scanf("%d", &n);
printf("Enter cost matrix:\n");
for(i=0; i<n; i++)
{
    for(j=0; j<n; j++)
    {
        scanf("%d", &a[i][j]);
    }
}
for(k=0; k<n; k++)
{
    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
        {
            if(a[i][j] > a[i][k] + a[k][j])
                a[i][j] = a[i][k] + a[k][j];
        }
    }
}
printf("Shortest Path Matrix:\n");
for(i=0; i<n; i++)
{
    for(j=0; j<n; j++)
    {
        printf("%d ", a[i][j]);
    }
    printf("\n");
}
getch();
}

```

Enter number of vertices: 4

Enter cost matrix:

0 5 999 10

999 0 3 999

999 999 0 1

999 999 999 0

Shortest Path Matrix:

0 5 8 9

999 0 3 4

999 999 0 1

999 999 999 0

15. FLOYD WARSHALL ALGORITHM

PSEUDOCODE

Input : Number of vertices n
Cost matrix $a[n][n]$
Output : All pairs shortest path matrix

Algorithm :

1. Read n
2. Read cost matrix $a[n][n]$
3. For $k = 0$ to $n-1$
4. For $i = 0$ to $n-1$
5. For $j = 0$ to $n-1$
6. If $a[i][j] > a[i][k] + a[k][j]$
7. $a[i][j] = a[i][k] + a[k][j]$
8. Print shortest path matrix $a[n][n]$

HOW IT WORKS

- Floyd-Warshall finds shortest paths between all pairs of vertices.
- It allows each vertex to be an intermediate vertex one by one.
- If going from i to j through k is shorter than current known path, update it.
- Works for both positive and negative weights (no negative cycles).

COMPLEXITY

- Time Complexity : $O(n^3)$
- Space Complexity : $O(n^2)$

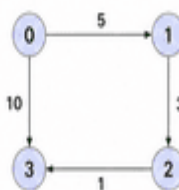
SAMPLE INPUT / OUTPUT

Input	Output
Enter number of vertices: 4	Shortest Path Matrix:
Enter cost matrix:	
0 5 999 10	0 5 8 9
999 0 3 999	999 0 3 4
999 999 0 1	999 999 0 1
999 999 999 0	999 999 999 0

FLOWCHART



EXAMPLE GRAPH



INPUT (COST MATRIX)

From \ To	0	1	2	3
0	0	5	999	10
1	999	0	3	999
2	999	999	0	1
3	999	999	999	0

ITERATIONS (k AS INTERMEDIATE VERTEX)

Initial Matrix (No intermediate vertex)					After k = 0 (Using vertex 0 as intermediate)				
From \ To	0	1	2	3	From \ To	0	1	2	3
0	0	5	999	10	0	0	5	999	10
1	999	0	3	999	1	999	0	3	999
2	999	999	0	1	2	999	1004	0	1
3	999	999	999	0	3	999	999	999	0

After k = 1 (Using vertex 1 as intermediate)					After k = 2 (Using vertex 2 as intermediate)				
From \ To	0	1	2	3	From \ To	0	1	2	3
0	0	5	8	10	0	0	5	8	9
1	999	0	3	999	1	999	0	3	4
2	999	1004	0	1	2	999	999	0	1
3	999	999	999	0	3	999	999	999	0

After k = 3, matrix remains the same.

SHORTEST PATH MATRIX (FINAL RESULT)

From \ To	0	1	2	3
0	0	5	8	9
1	999	0	3	4
2	999	999	0	1
3	999	999	999	0

Interpretation

- Shortest path from 0 to 2 is 8 (0 → 1 → 2)
- Shortest path from 0 to 3 is 9 (0 → 1 → 2 → 3)
- Shortest path from 1 to 3 is 4 (1 → 2 → 3)
- Others are shown in the matrix.

16. N Queen Problem

```
#include<stdio.h>

#include<conio.h>

#include<stdlib.h>

int x[10], count = 0;

int place(int k, int i)
{
    int j;
    for(j=1; j<k; j++)
    {
        if(x[j] == i || abs(x[j]- i) == abs(j- k))
            return 0;
    }
    return 1;
}

void nqueen(int k, int n)
{
    int i, j;
    for(i=1; i<=n; i++)
    {
        if(place(k, i))
        {
            x[k] = i;
            if(k == n)
            {
                count++;
                printf("\nSolution %d:\n", count);

                for(j=1; j<=n; j++)
                    printf("%d ", x[j]);
```

```

    }
    else
    {
        nqueen(k+1, n);
    }
}
}
}

```

```

void main(){algorithm

```

```

    int n;

    clrscr();

    printf("Enter number of queens: ");

    scanf("%d", &n);

    nqueen(1, n);

    getch();

}

```

Enter number of queens: 4

Solution 1:

2 4 1 3

Solution 2:

3 1 4 2

16. N QUEEN PROBLEM

PSEUDOCODE

Global: int x[10], count = 0

Function place(k, i)

```
1. for j = 1 to k - 1
2.   if (x[j] == i || abs(x[j] - i) == abs(j - k))
3.     return 0 // position (k, i) not safe
4. return 1 // position (k, i) safe
```

Procedure nqueen(k, n)

```
1. for i = 1 to n
2.   if place(k, i) == 1 then
3.     x[k] = i // place queen in row k, col i
4.     if k == n then // all queens placed
5.       count = count + 1
6.       print solution x[1..n]
7.     else
8.       nqueen(k + 1, n) // place next queen
```

Main

```
1. Read n
2. Call nqueen(1, n)
```

HOW IT WORKS

- Place queens row by row.
- For each row k, try all columns i from 1 to n.
- Use place(k, i) to check if queen can be placed at row k, column i safely.
- If safe, place the queen and move to next row.
- If k > n, print the solution.

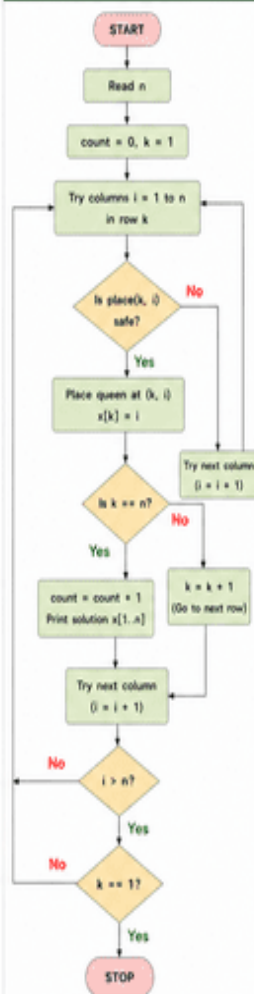
COMPLEXITY

- Time Complexity : $O(N!)$ (Backtracking)
- Space Complexity : $O(N)$ (Recursion stack + array)

SAMPLE INPUT / OUTPUT

Input	Output
Enter number of queens: 4	Solution 1: 2 4 1 3
	Solution 2: 3 1 4 2

FLOWCHART



EXAMPLE (N = 4)

Goal: Place 4 queens on a 4x4 chessboard such that no two queens attack each other.

SOLUTIONS

Solution 1:

2 4 1 3

x[i] = column number of queen in row i

(Row 1 → Col 2, Row 2 → Col 4,
Row 3 → Col 1, Row 4 → Col 3)

Solution 2:

3 1 4 2

x[i] = column number of queen in row i

(Row 1 → Col 3, Row 2 → Col 1,
Row 3 → Col 4, Row 4 → Col 2)

BOARD REPRESENTATION

Solution 1: 2 4 1 3

	1	2	3	4
1	.	Q	.	.
2	.	.	.	Q
3	Q	.	.	.
4	.	.	Q	.

Solution 2: 3 1 4 2

	1	2	3	4
1	.	.	Q	.
2	Q	.	.	.
3	.	.	.	Q
4	.	Q	.	.

SAFE POSITION CHECK (place(k, i))

A position (k, i) is safe if for all rows j < k:

- $x[j] \neq i$ → Same column check
- $abs(x[j] - i) \neq abs(j - k)$ → Diagonal check

Where x[j] is the column of queen in row j.

BACKTRACKING (EXAMPLE FOR N = 4)

- Start at row 1, try column 1 → place queen.
- Go to row 2 and try safe columns.
- If no safe column, backtrack to previous row and try next column.
- Continue until all 4 queens are placed and solutions are printed.

SAMPLE RUN

Input	Output
Enter number of queens: 4	Solution 1: 2 4 1 3
	Solution 2: 3 1 4 2

17. Hamiltonian Cycle

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int x[10],g[10][10],n;
```

```
void nextvalue(int k){
```

```
int j;
```

```
while(1){
```

```
    x[k]=(x[k]+1)%(n+1);
```

```
    if(x[k]==0)
```

```
        return;
```

```
    if(g[x[k-1]][x[k]]!=0){
```

```
        for(j=1;j<=k-1;j++){
```

```

        if(x[j]==x[k])
            break;
    }
    if(j==k){
        if((k<n) || (k==n && g[x[n]][x[1]]!=0))
            return;
    }
}
}
}

```

```

void hamiltonian(int k){
    while(1)
    {
        nextvalue(k);
        if(x[k]==0)
            return;
        if(k==n)
        {
            int i;
            printf("\nCycle: ");
            for(i=1;i<=n;i++)
                printf("%d ",x[i]);
            printf("%d",x[1]);
        }
        else
        {
            hamiltonian(k+1);
        }
    }
}
}

```

```

void main(){
    int i,j;
    clrscr();
    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter adjacency matrix:\n");
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            scanf("%d",&g[i][j]);
        }
    }
    for(i=1;i<=n;i++)
        x[i]=0;
    x[1]=1;
    hamiltonian(2);
    getch();
}

```

Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 1

1 0 1 0

1 1 0 1

1 0 1 0

Cycle: 1 2 3 4 1

Cycle: 1 4 3 2 1

17. HAMILTONIAN CYCLE

PSEUDOCODE

```
Global: int x[10], g[10][10], n
Procedure nextvalue(k)
1. while (1) do
2.   x[k] = (x[k] + 1) % (n + 1)
3.   if x[k] == 0 then return
4.   if g[x[k-1]][x[k]] != 0 then
5.     for j = 1 to k - 1 do
6.       if x[j] == x[k] then break
7.     if j == k then
8.       if (k < n) or (k == n and g[x[n]][x[1]] != 0)
9.         return
Procedure hamiltonian(k)
1. while (1) do
2.   nextvalue(k)
3.   if x[k] == 0 then return
4.   if k == n then
5.     print cycle x[1] x[2] ... x[n] x[1]
6.   else
7.     hamiltonian(k + 1)
Main
1. Read n
2. Read adjacency matrix g[n][n]
3. for i = 1 to n do x[i] = 0
4. x[1] = 1
5. hamiltonian(2)
```

HOW IT WORKS

- Fix the first vertex as 1.
- Generate permutations of the remaining vertices one by one using nextvalue(k).
- Check if the next vertex is adjacent to the previous vertex and is not already in the path.
- If all vertices are included (k == n) and there is an edge from last vertex to the first, print the cycle.
- Backtrack and try other possibilities.

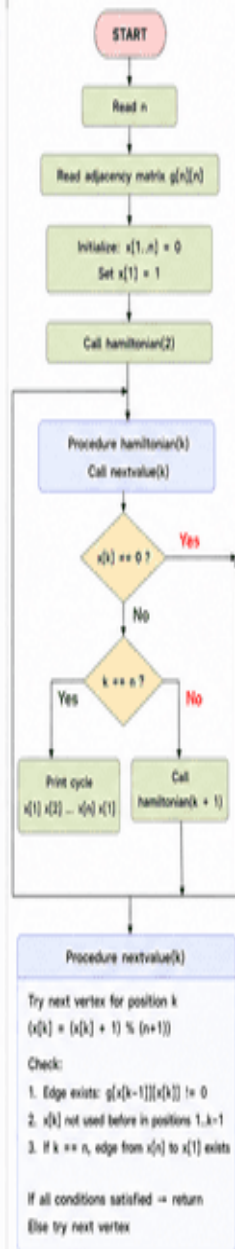
COMPLEXITY

- Time Complexity : $O(n!)$ (Worst Case)
- Space Complexity : $O(n^2)$ (Adjacency Matrix) + $O(n)$ (Recursion Stack)

SAMPLE INPUT / OUTPUT

Input	Output
Enter number of vertices: 4	Cycle: 1 2 3 4 1
Enter adjacency matrix:	Cycle: 1 4 3 2 1
0 1 1 1	
1 0 1 0	
1 1 0 1	
1 0 1 0	

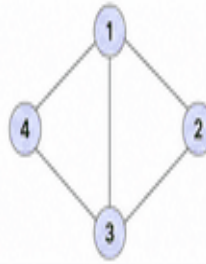
FLOWCHART



```

Procedure nextvalue(k)
Try next vertex for position k
x[k] = (x[k] + 1) % (n + 1)
Check:
1. Edge exists: g[x[k-1]][x[k]] != 0
2. x[k] not used before in positions 1..k-1
3. If k == n, edge from x[n] to x[1] exists
If all conditions satisfied == return
Else try next vertex
  
```

EXAMPLE GRAPH (n = 4)



ADJACENCY MATRIX (g[i][j])

i \ j	1	2	3	4
1	0	1	1	1
2	1	0	1	0
3	1	1	0	1
4	1	0	1	0

HOW THE ALGORITHM WORKS (Example for n = 4)

Step	k	Current Path (x[1..k])	Next Choices Tried	Remarks
1	1	1	2, 3, 4	Start at vertex 1
2	2	1 - 2	3, 4	2 is adjacent to 1
3	3	1 - 2 - 3	4	3 is adjacent to 2
4	4	1 - 2 - 3 - 4	-	4 is adjacent to 3 and 4 to 1 exists → print cycle 1 2 3 4 1
5	3	1 - 2	(backtrack)	No more choices for k=3, backtrack
6	2	1	4	Try next choice for k=2
7	3	1 - 4	3	3 is adjacent to 4
8	4	1 - 4 - 3 - 2	-	2 is adjacent to 3 and 2 to 1 exists → print cycle 1 4 3 2 1

VALID CHOICES IN nextvalue(k)

A vertex x[k] is valid if:

- $g[x[k-1]][x[k]] \neq 0$ → edge exists
- $x[k]$ not used in positions 1 to k-1 → no repetition
- If $k == n$, $g[x[n]][x[1]] \neq 0$ → cycle must close

Otherwise, try next vertex.

SAMPLE RUN

Input	Output
Enter number of vertices: 4	Cycle: 1 2 3 4 1
Enter adjacency matrix:	Cycle: 1 4 3 2 1
0 1 1 1	
1 0 1 0	
1 1 0 1	
1 0 1 0	

18. Matrix Chain Multiplication, Dynamic Programming Optimisation

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int p[10],m[10][10];
```

```
int i,j,k,n,q,len;
```

```

clrscr();

printf("Enter number of matrices: ");

scanf("%d",&n);

printf("Enter dimensions:\n");

for(i=0;i<=n;i++)

    scanf("%d",&p[i]);

for(i=1;i<=n;i++)

    m[i][i]=0;

for(len=2;len<=n;len++)

{

    for(i=1;i<=n-len+1;i++)

    {

        j=i+len-1;

        m[i][j]=9999;

        for(k=i;k<j;k++)

        {

            q=m[i][k]+m[k+1][j]+p[i-1]*p[k]*p[j];

            if(q<m[i][j])

                m[i][j]=q;

        }

    }

}

printf("Minimum multiplication cost = %d",m[1][n]);

getch();

}

```

Output

Enter number of matrices: 3

Enter dimensions:

10 20 30 40

Minimum multiplication cost = 18000

18. MATRIX CHAIN MULTIPLICATION (DYNAMIC PROGRAMMING OPTIMISATION)

PSEUDOCODE

Input : Number of matrices n
 Array $p[0..n]$ of dimensions
 If matrix A_i has dimension $p[i-1] \times p[i]$

Output : Minimum number of multiplications

1. Read n
2. Read $p[0..n]$
3. for $i = 1$ to n
4. $m[i][i] = 0$
5. for $len = 2$ to n
6. for $i = 1$ to $n - len + 1$
7. $j = i + len - 1$
8. $m[i][j] = 9999$
9. for $k = i$ to $j - 1$
10. $q = m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j]$
11. if $q < m[i][j]$
12. $m[i][j] = q$
13. Print $m[1][n]$ as minimum multiplication cost

HOW IT WORKS

- We have a chain of matrices $A_1, A_2, \dots, A_n$.
- A_i has dimension $p[i-1] \times p[i]$.
- The goal is to find the most efficient way to multiply the chain.
- $m[i][j]$ stores the minimum cost of multiplying matrices from A_i to A_j .
- We solve the problem for chains of length 1, then length 2, and so on up to n .

COMPLEXITY

- Time Complexity : $O(n^3)$
- Space Complexity : $O(n^2)$

SAMPLE INPUT / OUTPUT

Input	Output
Enter number of matrices: 3	Minimum multiplication cost = 18000
Enter dimensions: 10 20 30 40	

FLOWCHART



EXAMPLE

Number of matrices (n) = 3
 Dimensions:
 $p[0]$ $p[1]$ $p[2]$ $p[3]$
 10 20 30 40

MATRICES

A_1 : 10 x 20
 A_2 : 20 x 30
 A_3 : 30 x 40

DP TABLE $m[i][j]$

$i \setminus j$	1	2	3
1	0	6000	18000
2		0	24000
3			0

$m[i][j]$ = Minimum cost of multiplying matrices from A_i to A_j

CALCULATION STEPS

Length 1 ($i = j$)
 $m[1][1] = 0$, $m[2][2] = 0$, $m[3][3] = 0$

Length 2
 $i = 1, j = 2$
 $k = 1$: $q = m[1][1] + m[2][2] + p[0] * p[1] * p[2]$
 $= 0 + 0 + 10 * 20 * 30 = 6000 \rightarrow m[1][2] = 6000$
 $i = 2, j = 3$
 $k = 2$: $q = m[2][2] + m[3][3] + p[2] * p[3] * p[4]$
 $= 0 + 0 + 20 * 30 * 40 = 24000 \rightarrow m[2][3] = 24000$

Length 3
 $i = 1, j = 3$
 $k = 1$: $q = m[1][1] + m[2][3] + p[0] * p[1] * p[3]$
 $= 0 + 24000 + 10 * 20 * 40 = 32000$
 $k = 2$: $q = m[1][2] + m[3][3] + p[0] * p[2] * p[3]$
 $= 6000 + 0 + 10 * 30 * 40 = 18000 \rightarrow m[1][3] = 18000$

Minimum multiplication cost = $m[1][3] = 18000$

SAMPLE RUN

Input	Output
Enter number of matrices: 3	Minimum multiplication cost = 18000
Enter dimensions: 10 20 30 40	

19. Optimal Merge Pattern, Greedy Optimisation

```

#include<stdio.h>
#include<conio.h>
void sort(int a[], int n)
{
    int i, j, temp;

    for(i=0; i<n-1; i++)
    {
        for(j=i+1; j<n; j++)
        {
            if(a[i] > a[j])
            {
                temp = a[i];
                a[i] = a[j];
            }
        }
    }
}
    
```



```

        a[j] = temp;
    }
}
}
}

void main()
{
    int a[20], n, i, cost = 0, sum;
    clrscr();
    printf("Enter number of files: ");
    scanf("%d", &n);
    printf("Enter file sizes:\n");
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);

    while(n > 1)
    {
        sort(a, n);
        sum = a[0] + a[1];
        cost += sum;
        a[1] = sum;
        for(i=0; i<n-1; i++)
            a[i] = a[i+1];
        n--;
    }
    printf("Optimal Merge Cost = %d", cost);
    getch();
}

```

Output

Enter number of files: 4

Enter file sizes:

20 30 10 5

Optimal Merge Cost = 115

19. OPTIMAL MERGE PATTERN (GREEDY OPTIMISATION)

PSEUDOCODE

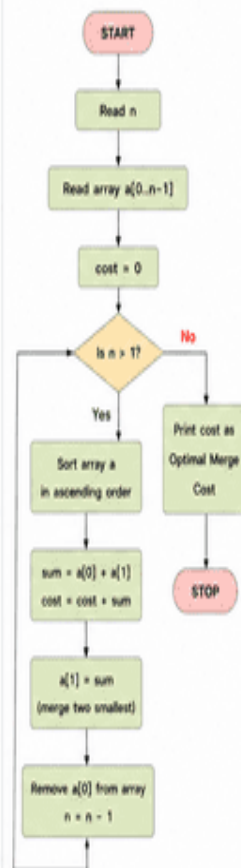
Input : Number of files n
 Array $a[0..n-1]$ of file sizes

Output : Minimum cost of merging all files

Algorithm :

1. Read n
2. Read array $a[0..n-1]$
3. $cost = 0$
4. While $n > 1$
 - a. Sort the array a in ascending order
 - b. $sum = a[0] + a[1]$ // merge two smallest
 - c. $cost = cost + sum$
 - d. $a[1] = sum$ // replace first two
 - e. Remove $a[0]$ from array
 - f. $n = n - 1$ // one file less
5. Print cost as Optimal Merge Cost

FLOWCHART



HOW IT WORKS

- At every step, pick the two smallest files and merge them.
- The cost of this merge is the sum of their sizes.
- Replace those two files with the merged file whose size is the sum.
- Repeat until only one file remains.
- The total cost is the Optimal Merge Cost.

COMPLEXITY

- Time Complexity : $O(n^2)$
 (due to sorting in each step)
- Space Complexity : $O(1)$ extra (in-place)

SAMPLE INPUT / OUTPUT

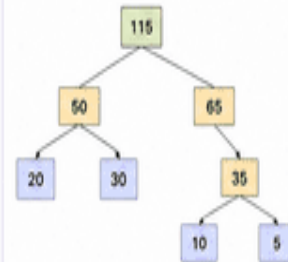
Input	Output
Enter number of files: 4	Optimal Merge Cost = 115
Enter file sizes: 20 30 10 5	

EXAMPLE

Number of files (n) = 4

File sizes: 20 30 10 5

MERGE PROCESS (MIN HEAP CONCEPT)



STEP BY STEP MERGE PROCESS

Step	Files (Sorted)	Pick Two Smallest	Merge Cost (sum)	Total Cost (so far)	Files After Merge
1	5 10 20 30	5 and 10	15	15	15 20 30
2	15 20 30	15 and 20	35	15 + 35 = 50	30 35
3	30 35	30 and 35	65	50 + 65 = 115	65
4	65	-	-	115	-

Optimal Merge Cost = 115

EXPLANATION

1. Merge 5 and 10 → cost = 15 → New list: 15, 20, 30
2. Merge 15 and 20 → cost = 35 → New list: 30, 35
3. Merge 30 and 35 → cost = 65 → New list: 65
4. Only one file remains. Stop.

Total Optimal Merge Cost = 15 + 35 + 65 = 115

SAMPLE RUN

Input	Output
Enter number of files: 4	Optimal Merge Cost = 115
Enter file sizes: 20 30 10 5	

20. BFS, Graph Traversal

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int a[10][10], visited[10], queue[10];
```

```
int front=-1,rear=-1;
```

```
void bfs(int v,int n)
```

```
{
    int i;
    for(i=0;i<n;i++)
    {
        if(a[v][i]==1 && visited[i]==0)
        {
            rear++;
            queue[rear]=i;
        }
    }
}
```

```

        visited[i]=1;
    }
}

if(front<=rear)
{
    printf("%d ",queue[front]+1);
    bfs(queue[front++],n);
}
}

void main()
{
    int n, i,j, start;
    clrscr();
    printf("Enter number of vertices: ");
    scanf("%d",&n);
    printf("Enter adjacency matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }

    for(i=0;i<n;i++)
        visited[i]=0;
    printf("Enter starting vertex: ");
    scanf("%d",&start);
    front=rear=0;
    queue[rear]=start-1;
    visited[start-1]=1;
    printf("BFS Traversal: ");
    printf("%d ",start);
    bfs(start-1,n);
    getch();
}

```

Output

Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 0

1 0 1 1

1 1 0 0

0 1 0 0

Enter starting vertex: 1

BFS Traversal: 1 2 3 4

20. BFS (BREADTH FIRST SEARCH) GRAPH TRAVERSAL

PSEUDOCODE

```

Global : a[10][10], visited[10], queue[10]
front = -1, rear = -1

Procedure bfs(v, n)
1. for i = 0 to n - 1
2.   if a[v][i] == 1 and visited[i] == 0
3.     rear = rear + 1
4.     queue[rear] = i
5.     visited[i] = 1
6. if front <= rear
7.   print queue[front] + 1
8.   bfs(queue[front++], n)

Main
1. Read n
2. Read adjacency matrix a[n][n]
3. for i = 0 to n - 1
4.   visited[i] = 0
5. Read starting vertex start
6. front = rear = 0
7. queue[rear] = start - 1
8. visited[start - 1] = 1
9. Print start
10. Call bfs(start - 1, n)
          
```

HOW IT WORKS

- BFS visits vertices level by level.
- Start from the given vertex.
- Visit all adjacent unvisited vertices and add them to the queue.
- Dequeue a vertex, visit it, and repeat the process for its adjacent vertices.
- Continue until the queue is empty.

COMPLEXITY

- Time Complexity : $O(n^2)$ (Adjacency Matrix)
- Space Complexity : $O(n)$ (Queue + Visited Array)

SAMPLE INPUT / OUTPUT

Input	Output
Enter number of vertices: 4	BFS Traversal: 1 2 3 4
Enter adjacency matrix:	
0 1 1 0	
1 0 1 1	
1 1 0 0	
0 1 0 0	
Enter starting vertex: 1	

FLOWCHART

```

graph TD
    Start([START]) --> ReadN[Read n]
    ReadN --> ReadAdj[Read adjacency matrix a[n][n]]
    ReadAdj --> InitVis[for i = 0 to n - 1  
visited[i] = 0]
    InitVis --> ReadStart[Read starting vertex start]
    ReadStart --> InitQueue[front = rear = 0  
queue[rear] = start - 1  
visited[start - 1] = 1]
    InitQueue --> PrintStart[Print start]
    PrintStart --> Decision{front <= rear?}
    Decision -- No --> Stop([STOP])
    Decision -- Yes --> Dequeue[front = front + 1  
Call bfs(queue[front], n)]
    Dequeue --> Decision
          
```

GRAPH (n = 4)

ADJACENCY MATRIX (a[i][j])

i \ j	1	2	3	4
1	0	1	1	0
2	1	0	1	1
3	1	1	0	0
4	0	1	0	0

BFS TRAVERSAL STEPS (Starting from vertex 1)

Step	Queue (front → rear)	Visited (1 2 3 4)	Dequeue / Visit	Enqueue New Adjacent Vertices	Traversal Order
0	1	1 0 0 0	Print 1	2, 3	1
1	2 3	1 1 1 0	Print 2	4	1 2
2	3 4	1 1 1 1	Print 3	-	1 2 3
3	4	1 1 1 1	Print 4	-	1 2 3 4

EXPLANATION

- Start from vertex 1.
- Visit all its adjacent vertices (2, 3) and add them to the queue.
- Dequeue 2, visit it, and add its unvisited adjacent vertex (4).
- Dequeue 3, no new vertex to add.
- Dequeue 4, no new vertex to add.
- Queue becomes empty. BFS is complete.

SAMPLE RUN

Input	Output
Enter number of vertices: 4	BFS Traversal: 1 2 3 4
Enter adjacency matrix:	
0 1 1 0	
1 0 1 1	
1 1 0 0	
0 1 0 0	
Enter starting vertex: 1	

21. DFS, Graph Traversal

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int a[10][10], visited[10], n;
```

```
void dfs(int v)
```

```
{
```

```
    int i;
```

```
    visited[v]=1;
```

```
    printf("%d ",v+1);
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        if(a[v][i]==1 && visited[i]==0)
```

```
            dfs(i);
```

```
    }
```

```

}

void main()
{
    int i,j,start;

    clrscr();

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter adjacency matrix:\n");

    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }

    for(i=0;i<n;i++)
        visited[i]=0;

    printf("Enter starting vertex: ");
    scanf("%d",&start);

    printf("DFS Traversal: ");

    dfs(start-1);

    getch();
}

```

Output

Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 0

1 0 1 1

1 1 0 0

0 1 0 0

Enter starting vertex: 1

DFS Traversal: 1 2 3 4

21. DFS (DEPTH FIRST SEARCH) GRAPH TRAVERSAL

PSEUDOCODE

Global : a[10][10], visited[10], n

Procedure dfs(v)

```
1. visited[v] = 1
2. print v + 1
3. for i = 0 to n - 1
4.     if a[v][i] == 1 and visited[i] == 0
5.         call dfs(i)
```

Main

```
1. read n
2. read adjacency matrix a[n][n]
3. for i = 0 to n - 1
4.     visited[i] = 0
5. read starting vertex start
6. print "DFS Traversal: "
7. call dfs(start - 1)
```

HOW IT WORKS

- Start from the given starting vertex.
- Mark the current vertex as visited and print it.
- Visit the first adjacent unvisited vertex and repeat the process.
- Continue until all reachable vertices are visited.
- This process follows depth before exploring other branches.

COMPLEXITY

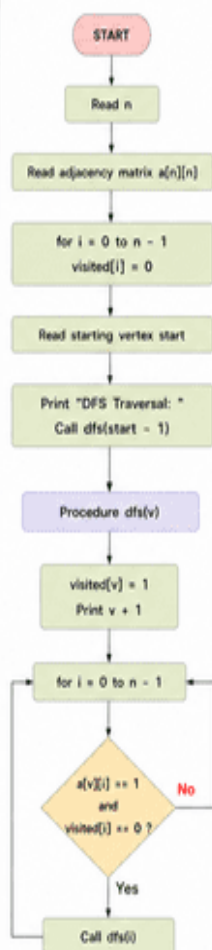
- Time Complexity : $O(n^2)$ (Adjacency Matrix)
- Space Complexity : $O(n)$ (Visited Array + Recursion Stack)

SAMPLE INPUT / OUTPUT

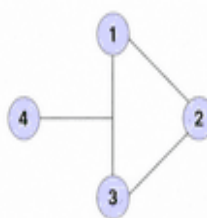
Input
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
1 0 1 1
1 1 0 0
0 1 0 0
Enter starting vertex: 1

Output
DFS Traversal:
1 2 3 4

FLOWCHART



GRAPH (n = 4)



ADJACENCY MATRIX (a[i][j])

i \ j	1	2	3	4
1	0	1	1	0
2	1	0	1	1
3	1	1	0	0
4	0	1	0	0

DFS TRAVERSAL STEPS (Starting from vertex 1)

Step	Current Vertex	Visited Array (1 2 3 4)	Action / Explanation	Traversal Order
1	1	1 0 0 0	Visit 1. Go to first adjacent vertex 2.	1
2	2	1 1 0 0	Visit 2. Go to first adjacent vertex 3.	1 2
3	3	1 1 1 0	Visit 3. No unvisited adjacent vertex. Backtrack to 2.	1 2 3
4	2 (Backtrack)	1 1 1 0	Next unvisited adjacent vertex of 2 is 4.	1 2 3 4
5	4	1 1 1 1	Visit 4. No unvisited adjacent vertex. Backtrack. Done.	1 2 3 4

EXPLANATION

- Start at vertex 1.
- From 1, visit 2 (first unvisited adjacent).
- From 2, visit 3 (first unvisited adjacent).
- From 3, no unvisited adjacent vertex. Backtrack to 2.
- From 2, next unvisited adjacent is 4. Visit 4.
- From 4, no unvisited adjacent vertex. Backtrack. DFS complete.

SAMPLE RUN

Input
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
1 0 1 1
1 1 0 0
0 1 0 0
Enter starting vertex: 1

Output
DFS Traversal: 1 2 3 4

22. Travelling Salesman Problem, Branch and Bound

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int a[10][10], visited[10], n, cost = 0;
```

```
void tsp(int city)
```

```
{
    int i, next;
    int min = 999;
    visited[city] = 1;
    printf("%d -> ", city + 1);
    for(i=0; i<n; i++)
    {
        if(a[city][i] != 0 && visited[i] == 0)
        {
            if(a[city][i] < min)
```

```

        {
            min = a[city][i];
            next = i;
        }
    }
}

if(min == 999)
{
    printf("1");
    cost += a[city][0];
    return;
}
cost += min;
tsp(next);
}

void main()
{
    int i, j;
    clrscr();
    printf("Enter number of cities: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
    for(i=0; i<n; i++)
        visited[i] = 0;
    printf("Path: ");
    tsp(0);
    printf("\nMinimum Cost = %d", cost);
    getch();
}

```

Output

Enter number of cities: 4

Enter cost matrix:

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Path: 1 -> 2 -> 4 -> 3 -> 1

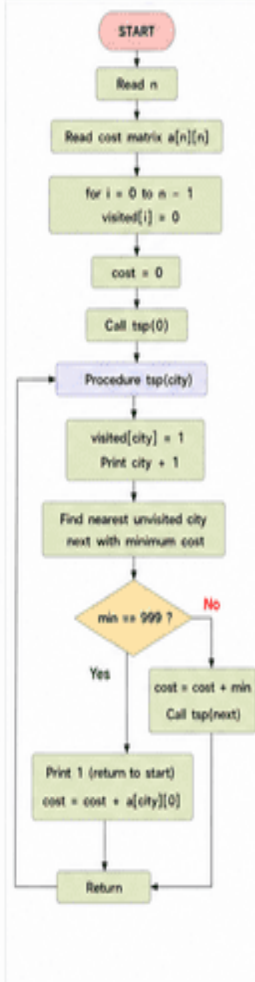
Minimum Cost = 80

22. TRAVELLING SALESMAN PROBLEM (BRANCH AND BOUND)

PSEUDOCODE

```
Global : a[10][10], visited[10], n, cost = 0
Procedure tsp(city)
1. visited[city] = 1
2. print city + 1
3. min = 999
4. for i = 0 to n - 1
5.   if a[city][i] != 0 and visited[i] == 0
6.     if a[city][i] < min
7.       min = a[city][i]
8.       next = i
9. if min == 999
10.  print 1
11.  cost = cost + a[city][0]
12.  return
13. cost = cost + min
14. tsp(next)
Main
1. Read n
2. Read cost matrix a[n][n]
3. for i = 0 to n - 1
4.   visited[i] = 0
5. Print "Path: "
6. Call tsp(0)
7. Print "Minimum Cost = " cost
```

FLOWCHART



HOW IT WORKS

- Start from city 1.
- From the current city, go to the nearest unvisited city.
- Repeat until all cities are visited.
- Finally, return to the starting city (1).
- The total cost is the sum of all selected edges.

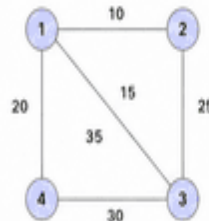
COMPLEXITY

- Time Complexity : $O(n^2)$
- Space Complexity : $O(n)$

SAMPLE INPUT / OUTPUT

Input	Output
Enter number of cities: 4	Path:
Enter cost matrix:	1 → 2 → 4 → 3 → 1
0 10 15 20	Minimum Cost = 80
10 0 35 25	
15 35 0 30	
20 25 30 0	

CITIES (n = 4)



COST MATRIX a[i][j]

i \ j	1	2	3	4
1	0	10	15	20
2	10	0	35	25
3	15	35	0	30
4	20	25	30	0

STEP BY STEP EXECUTION (Nearest Unvisited City)

Step	Current City	Unvisited Cities	Chosen Next City	Cost Added	Total Cost	Path So Far
1	1	{2, 3, 4}	2 (cost 10)	10	10	1 → 2
2	2	{3, 4}	4 (cost 25)	25	35	1 → 2 → 4
3	4	{3}	3 (cost 30)	30	65	1 → 2 → 4 → 3
4	3	{ } (None)	Return to 1 (cost 15)	15	80	1 → 2 → 4 → 3 → 1

EXPLANATION

- From city 1, nearest unvisited city is 2 (cost 10).
- From city 2, nearest unvisited city is 4 (cost 25).
- From city 4, nearest unvisited city is 3 (cost 30).
- All cities visited. Return to city 1 from city 3 (cost 15).
- Total minimum cost = 10 + 25 + 30 + 15 = 80.

SAMPLE RUN

Input (as per program)	Output (as per program)
Enter number of cities: 4	Path: 1 → 2 → 4 → 3 → 1
Enter cost matrix:	Minimum Cost = 80
0 10 15 20	
10 0 35 25	
15 35 0 30	
20 25 30 0	